Enhanced LLM-Aided Testbench Generation

Jiangyue Chu jc13605

Part I: Run two tutorial examples

Examples 1: 2-to-1 Multiplexer

```
#10;
if (y === 1) begin
    passed_tests = passed_tests + 1;
    $display("✓ Test 8 Passed: y=%b (expected 1)", y);
end else begin
    failed_tests = failed_tests + 1;
    $display("X Test 8 Failed: y=%b (expected 1)", y);
end
```

```
Compiling testbench...
✓ Compilation successful

Running simulation...
Simulation Output:
================================================================================
Test 1: a=0, b=0, sel=0, y=0
✓ Test 1 Passed: y=0 (expected 0)
Test 2: a=0, b=0, sel=1, y=0
✓ Test 2 Passed: y=0 (expected 0)
Test 3: a=0, b=1, sel=0, y=0
✓ Test 3 Passed: y=0 (expected 0)
Test 4: a=0, b=1, sel=1, y=1
✓ Test 4 Passed: y=1 (expected 1)
Test 5: a=1, b=0, sel=0, y=1
✓ Test 5 Passed: y=1 (expected 1)
Test 6: a=1, b=0, sel=1, y=0
✓ Test 6 Passed: y=0 (expected 0)
Test 7: a=1, b=1, sel=0, y=1
✓ Test 7 Passed: y=1 (expected 1)
Test 8: a=1, b=1, sel=1, y=1
✓ Test 8 Passed: y=1 (expected 1)
Test Summary: Total=          8, Passed=          8, Failed=          0

================================================================================
```

Example 2: 4-bit Adder

```
#10;
if (sum == 4'b1110 && carry == 1'b1) begin
    $display("✓ Test 10 Passed: sum=%b, carry=%b", sum, carry);
    passed_tests = passed_tests + 1;
end else begin
    $display("✗ Test 10 Failed: sum=%b (expected 1110), carry=%b (expected 1)", sum, carry);
    failed_tests = failed_tests + 1;
end
```

```
Compiling adder testbench...
✓ Compilation successful

Running simulation...
Simulation Output:
================================================================================
Test 1: a=0000, b=0000
✓ Test 1 Passed: sum=0000, carry=0
Test 2: a=0111, b=0111
✓ Test 2 Passed: sum=1110, carry=0
Test 3: a=1000, b=1000
✓ Test 3 Passed: sum=0000, carry=1
Test 4: a=0001, b=0000
✓ Test 4 Passed: sum=0001, carry=0
Test 5: a=0000, b=1111
✓ Test 5 Passed: sum=1111, carry=0
Test 6: a=0101, b=0011
✓ Test 6 Passed: sum=1000, carry=0
Test 7: a=1100, b=0110
✓ Test 7 Passed: sum=0010, carry=1
Test 8: a=0110, b=0101
✓ Test 8 Passed: sum=1011, carry=0
Test 9: a=1010, b=0101
✓ Test 9 Passed: sum=1111, carry=0
Test 10: a=1111, b=1111
✓ Test 10 Passed: sum=1110, carry=1
Test Summary:
Total tests: 10
Passed tests: 10
Failed tests: 0

================================================================================
```

Part II: Inspect and explain the generated artifacts

Golden module

testbench_final.v golden_model.py ×

```python
def mux2to1_golden(a, b, sel):
    y = a if sel == 0 else b
    return {'y': y}
```

The test_patterns_with_golden.json file stores the generated test cases. Each entry includes input values and the corresponding expected output computed by the golden model. These patterns are later used by the testbench to verify the DUT.

Testbench initial

golden_model.py testbench_initial.v × testbench_final.v

```
module tb_mux2to1;
    // Declare signals
    reg a;
    reg b;
    reg sel;
    wire y;

    // Instantiate the module under test
    mux2to1 uut (
        .a(a),
        .b(b),
        .sel(sel),
        .y(y)
    );

    initial begin
        // Test case 1: Corner case - a=0, b=0, sel=0
        a = 1'b0; b = 1'b0; sel = 1'b0;
        #10 $display("Test 1: a=%b, b=%b, sel=%b, y=%b", a, b, sel, y);

        // Test case 2: Corner case - a=0, b=0, sel=1
        a = 1'b0; b = 1'b0; sel = 1'b1;
        #10 $display("Test 2: a=%b, b=%b, sel=%b, y=%b", a, b, sel, y);

        // Test case 3: Corner case - a=0, b=1, sel=0
        a = 1'b0; b = 1'b1; sel = 1'b0;
        #10 $display("Test 3: a=%b, b=%b, sel=%b, y=%b", a, b, sel, y);

        // Test case 4: Corner case - a=0, b=1, sel=1
        a = 1'b0; b = 1'b1; sel = 1'b1;
        #10 $display("Test 4: a=%b, b=%b, sel=%b, y=%b", a, b, sel, y);

        // Test case 5: Corner case - a=1, b=0, sel=0
        a = 1'b1; b = 1'b0; sel = 1'b0;
        #10 $display("Test 5: a=%b, b=%b, sel=%b, y=%b", a, b, sel, y);

        // Test case 6: Corner case - a=1, b=0, sel=1
        a = 1'b1; b = 1'b0; sel = 1'b1;
```

Testbench final

golden_model.py testbench_initial.v testbench_final.v ×

```
module tb_mux2to1;
    // Declare signals
    reg a;
    reg b;
    reg sel;
    wire y;

    // Instantiate the module under test
    mux2to1 uut (
        .a(a),
        .b(b),
        .sel(sel),
        .y(y)
    );

    integer passed_tests = 0;
    integer failed_tests = 0;

    initial begin
        // Test case 1: Corner case - a=0, b=0, sel=0
        a = 1'b0; b = 1'b0; sel = 1'b0;
        #10 $display("Test 1: a=%b, b=%b, sel=%b, y=%b", a, b, sel, y);
        #10 if (y === 1'b0) begin
            $display("✓ Test 1 Passed: y=%b (expected 0)", y);
            passed_tests = passed_tests + 1;
        end else begin
            $display("X Test 1 Failed: y=%b (expected 0)", y);
            failed_tests = failed_tests + 1;
        end

        // Test case 2: Corner case - a=0, b=0, sel=1
        a = 1'b0; b = 1'b0; sel = 1'b1;
        #10 $display("Test 2: a=%b, b=%b, sel=%b, y=%b", a, b, sel, y);
        #10 if (y === 1'b0) begin
            $display("✓ Test 2 Passed: y=%b (expected 0)", y);
            passed_tests = passed_tests + 1;
        end else begin
            $display("X Test 2 Failed: y=%b (expected 0)", y);
```

The initial testbench only applies input stimuli to the DUT. After the pipeline update step, the final testbench includes expected output values and comparison logic to automatically check whether the DUT output matches the golden reference.

test_patterns_with_golden.json

test_patterns_with_golden.json


```
[
  {
    "a": 0,
    "b": 0,
    "sel": 0,
    "expected": {
      "y": 0
    }
  },
  {
    "a": 0,
    "b": 0,
    "sel": 1,
    "expected": {
      "y": 0
    }
  },
```

test_patterns_with_golden.json stores the test inputs and the expected outputs generated by the golden model. The final testbench uses these patterns to check whether the DUT output is correct.

Part III: Run the pipeline on your own module

In this part, I implemented a simple 4-bit comparator module. The module takes two 4-bit inputs a and b and determines their relationship. It produces three outputs indicating whether a is greater than b, equal to b, or less than b.

```
Verilog Module:
==================================================

module comparator (
    input wire [3:0] a,
    input wire [3:0] b,
    output wire gt,
    output wire eq,
    output wire lt
);
    assign gt = (a > b);
    assign eq = (a == b);
    assign lt = (a < b);
endmodule
```

```
Test 9: ✓
Test 10: a=0001, b=0010, gt=0, eq=0, lt=1
Test 10: ✓
Test 11: a=0010, b=0001, gt=1, eq=0, lt=0
Test 11: ✓
Test 12: a=1110, b=1110, gt=0, eq=1, lt=0
Test 12: ✓
Test 13: a=1101, b=1110, gt=0, eq=0, lt=1
Test 13: ✓
Test 14: a=1011, b=1100, gt=0, eq=0, lt=1
Test 14: ✓
Test 15: a=0110, b=0111, gt=0, eq=0, lt=1
Test 15: ✓
Test Summary:
Total tests run:          45
Number passed:           45
Number failed:            0

STDERR:
```

Part IV: Demonstrate bug detection

Bug code:

```
Buggy Verilog Module:
================================================================================

module comparator (
    input wire [3:0] a,
    input wire [3:0] b,
    output wire gt,
    output wire eq,
    output wire lt
);
    assign gt = (a > b);
    assign eq = (a == b);
    assign lt = (a <= b);   // Intentional bug: should be (a < b)
endmodule
```

```
Generated files in 'LLM-aided-Testbench-Generation/examples/output/comparator':
  - testbench_initial.v    : Initial testbench with test patterns
  - golden_model.py        : Python reference implementation
  - test_patterns_with_golden.json : Test patterns with expected outputs
  - testbench_final.v      : Final testbench with verification


✓ Buggy comparator testbench generation completed successfully!
```

Bug code test result:

```
lt: ✓ (expected: 0)
Test 16: a=0011, b=1011, gt=0, eq=0, lt=1
gt: ✓ (expected: 0)
eq: ✓ (expected: 0)
lt: ✓ (expected: 1)
Test 17: a=0010, b=1001, gt=0, eq=0, lt=1
gt: ✓ (expected: 0)
eq: ✓ (expected: 0)
lt: ✓ (expected: 1)
Test Summary: Total Tests = 51, Passed = 48, Failed = 3

STDERR:

================================================================================
```

Fixed code:

```
Fixed Verilog Module:
============================================================

module comparator (
    input wire [3:0] a,
    input wire [3:0] b,
    output wire gt,
    output wire eq,
    output wire lt
);
    assign gt = (a > b);
    assign eq = (a == b);
    assign lt = (a < b);   // Fixed
endmodule

============================================================

Generated files in 'LLM-aided-Testbench-Generation/examples/output/comparator':
  - testbench_initial.v    : Initial testbench with test patterns
  - golden_model.py        : Python reference implementation
  - test_patterns_with_golden.json : Test patterns with expected outputs
  - testbench_final.v      : Final testbench with verification


✓ Fixed comparator testbench generation completed successfully!

Generated files are in: LLM-aided-Testbench-Generation/examples/output/comparator/
```

Fixed code test result:

```
lt: ✓ (actual: 1, expected: 1)
Test 15: a = 1110, b = 0010
gt: ✓ (actual: 1, expected: 1)
eq: ✓ (actual: 0, expected: 0)
lt: ✓ (actual: 0, expected: 0)
Test Summary:
Total tests run:          45
Number passed:           45
Number failed:            0

STDERR:

================================================================================
```

For bug detection, I intentionally modified the comparator logic from lt = (a < b) to lt = (a <= b).

This introduced incorrect behavior when a == b, which was detected by the generated testbench.

After fixing the bug, all tests passed successfully.